3. Front-end com react

Sumário

3. Front-end com react 1
Módulo 2 - Criando um Projeto React com Vite 2
- Instalando o Vite e Criando um Projeto 2
- Estrutura Básica de um Projeto React 9
- Componentizando elementos 10

Módulo 2 - Criando um Projeto React com Vite

➢ Instalando o Vite e Criando um Projeto

A ferramenta de construção utilizada pela equipe de desenvolvimento web permite a criação de projetos em diferentes frameworks e bibliotecas, como React, Vue, Svelte, entre outros. Além disso, os projetos criados utilizando o Vite são mais leves e, portanto, carregam mais rapidamente.

Para instalar o Vite, abra o terminal na pasta em que deseja criar o projeto e digite um dos comandos abaixo:

```
npm create vite@latest
```

ou

```
yarn create vite
```

Tanto o npm quanto o yarn são gerenciadores de pacotes. Ao escolher um deles, instale, remova ou atualize pacotes sempre utilizando os comandos do gerenciador escolhido.

Antes de tentar criar o projeto usando o yarn, verifique se ele está instalado na máquina com o seguinte comando no terminal:

```
yarn --version
```

Caso seja exibida a versão, o yarn está instalado. Caso contrário, execute o comando abaixo para instalá-lo globalmente:

```
npm install --global yarn
```

A tabela a seguir apresenta os principais comandos do npm e do yarn, bem como sua utilização:

NPM	YARN	Utilização
npm init	yarn init	inicializar o gerenciador
npm install -pacote-	yarn add -pacote-	instalar um pacote no projeto
npm update -pacote-	yarn upgrade -pacote-	atualizar um pacote do projeto
npm remove -pacote-	yarn remove -pacote-	remover um pacote do

		projeto

Com os pré-requisitos verificados, siga o passo a passo abaixo para criar o projeto utilizando o Vite. Neste exemplo, é utilizado o gerenciador de pacotes npm, mas o yarn também pode ser utilizado.

A seguir, é apresentado o passo a passo para a criação de um projeto utilizando o Vite. Neste exemplo, é utilizado o gerenciador de pacotes npm, mas o yarn também pode ser utilizado.

1. Dentro da pasta escolhida para armazenar o projeto, clique com o botão direito do mouse e selecione a opção Abrir no Terminal, conforme ilustrado na Imagem 5.

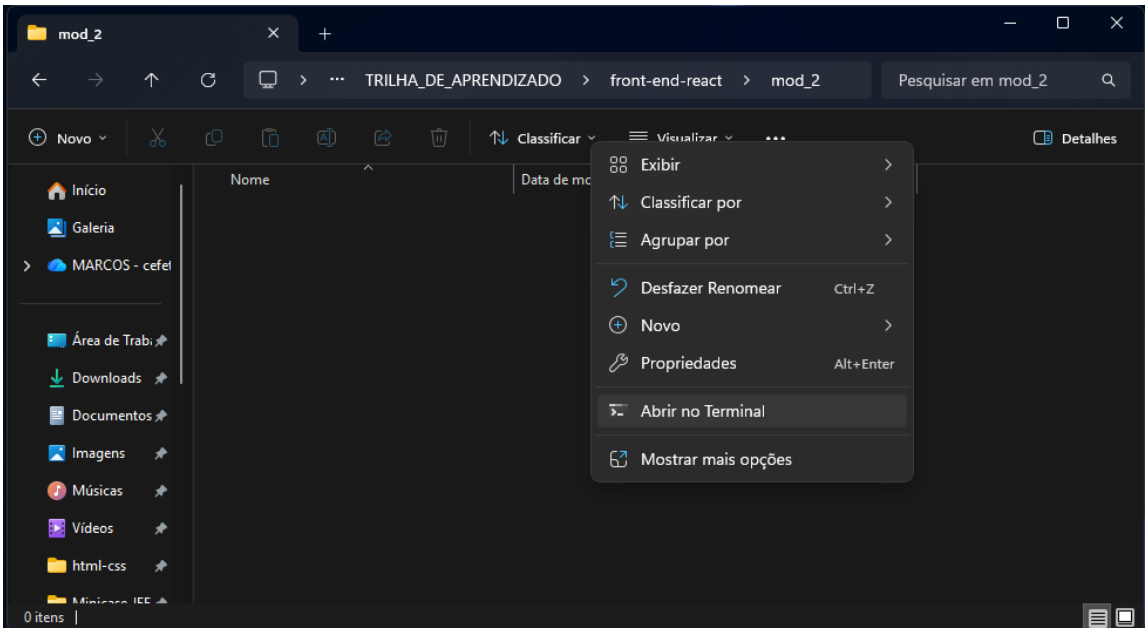

Imagem 5: Abrir pasta no terminal

2. No terminal, digite o comando de criação do projeto `npm create vite@latest` e pressione Enter e informe o nome do projeto, conforme ilustrado na Imagem 6.

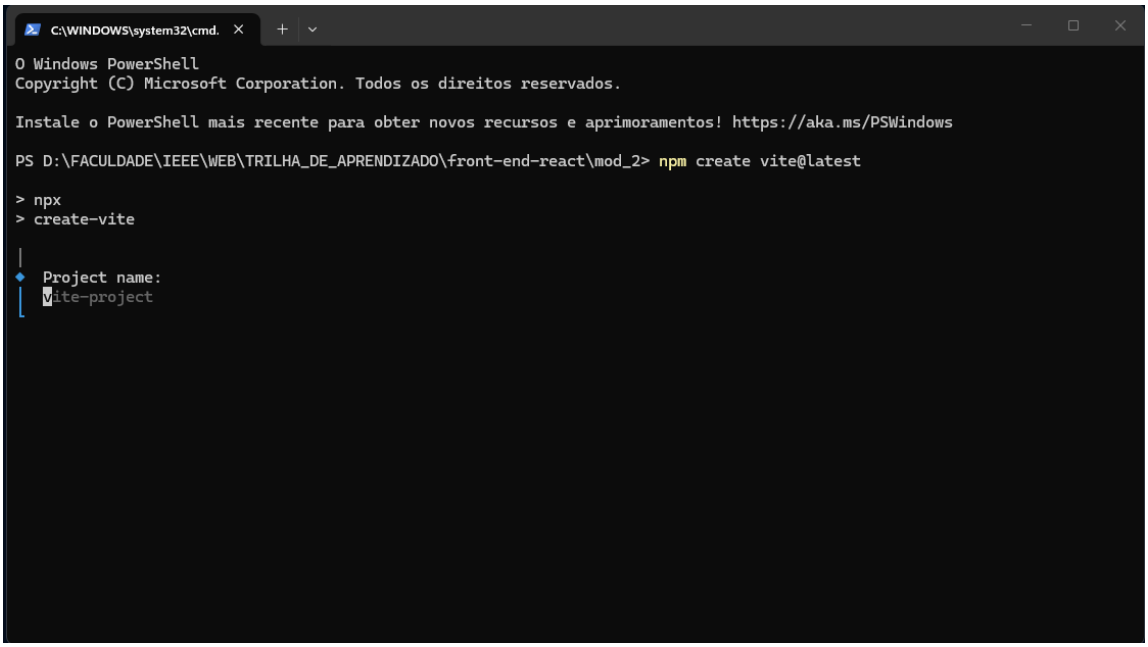

Imagem 6: Criar e nomear projeto

3. Escolha o framework ou biblioteca que será utilizada no projeto, neste caso, React, conforme ilustrado na Imagem 7.

```
C:\WINDOWS\system32\cmd.

O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS D:\FACULDADE\IEEE\WEB\TRILHA_DE_APRENDIZADO\front-end-react\mod_2> npm create vite@latest

> npx
> create-vite

◇  Project name:
│  projeto-mod2-aula1
◆  Select a framework:
│  ● Vanilla
│  ○ Vue
│  ○ React
│  ○ Preact
│  ○ Lit
│  ○ Svelte
│  ○ Solid
│  ○ Qwik
│  ○ Angular
│  ○ Marko
│  ○ Others
```

Imagem 7: Escolha do framework

4. Selecione a linguagem que será utilizada. Neste exemplo, escolha JavaScript, conforme ilustrado na Imagem 8.

```
C:\WINDOWS\system32\cmd.

Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS D:\FACULDADE\IEEE\WEB\TRILHA_DE_APRENDIZADO\front-end-react\mod_2> npm create vite@latest

> npx
> create-vite

◇  Project name:
│  projeto-mod2-aula1
◇  Select a framework:
│  React
◆  Select a variant:
│  ● TypeScript
│  ○ TypeScript + React Compiler
│  ○ TypeScript + SWC
│  ○ JavaScript
│  ○ JavaScript + React Compiler
│  ○ JavaScript + SWC
│  ○ React Router v7 ↗
│  ○ TanStack Router ↗
│  ○ RedwoodSDK ↗
│  ○ RSC ↗
│  ○ Vike ↗
```

Imagem 8: Escolha da linguagem

5. Na opção rollup-vite, selecione não, conforme ilustrado na Imagem 9.

```
C:\WINDOWS\system32\cmd.

Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS D:\FACULDADE\IEEE\WEB\TRILHA_DE_APRENDIZADO\front-end-react\mod_2> npm create vite@latest

> npx
> create-vite

◇  Project name:
│  projeto-mod2-aula1
◇  Select a framework:
│  React
◇  Select a variant:
│  JavaScript
◆  Use rolldown-vite (Experimental)?:
│  ○ Yes
│  ● No
```

Imagem 9: Opção rollup-vite

6. Quando solicitado, escolha sim para instalar as dependências com o npm e iniciar o projeto, conforme ilustrado na Imagem 10.

```
C:\WINDOWS\system32\cmd.

Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS D:\FACULDADE\IEEE\WEB\TRILHA_DE_APRENDIZADO\front-end-react\mod_2> npm create vite@latest

> npx
> create-vite

◇  Project name:
│  projeto-mod2-aula1
◇  Select a framework:
│  React
◇  Select a variant:
│  JavaScript
◇  Use rolldown-vite (Experimental)?:
│  No
◆  Install with npm and start now?
│  ● Yes / ○ No
```

Imagem 10: Instalar e iniciar projeto

7. Após a conclusão das instalações, o terminal exibirá a URL em que o projeto está rodando. Pressione a tecla Ctrl e clique no link exibido para abrir o projeto no navegador, conforme ilustrado na Imagem 11.

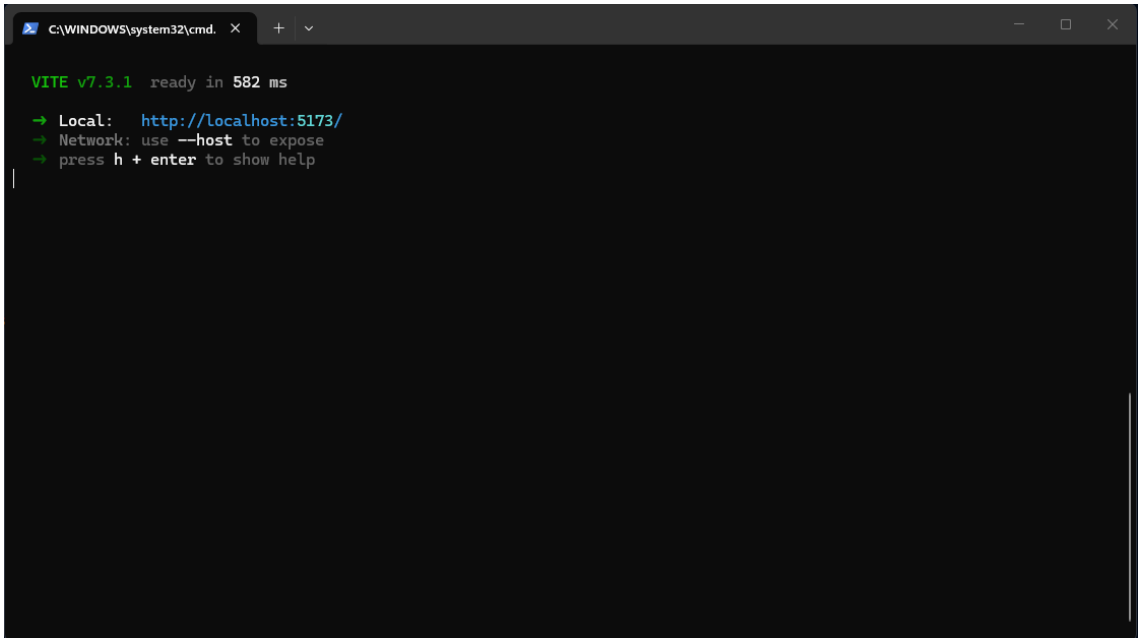

Imagem 11: Abrir projeto no navegador

8. O projeto será aberto no navegador, conforme ilustrado na Imagem 12.

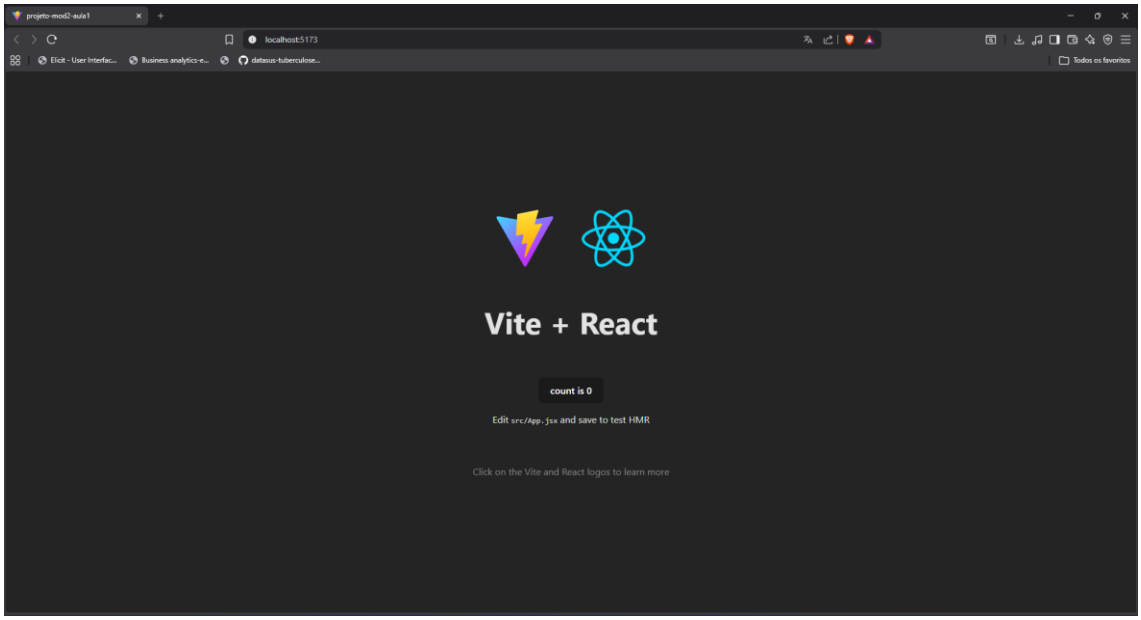

Imagem 12: Projeto no navegador

Após confirmar que o projeto foi criado corretamente e está em execução no navegador, o próximo passo é abrir a pasta do projeto em um editor de código para iniciar o desenvolvimento.

1. Para isso, clique com o botão direito na pasta do projeto e selecione a opção Abrir no Terminal. Em seguida, digite o comando abaixo:

   ```
   code .
   ```

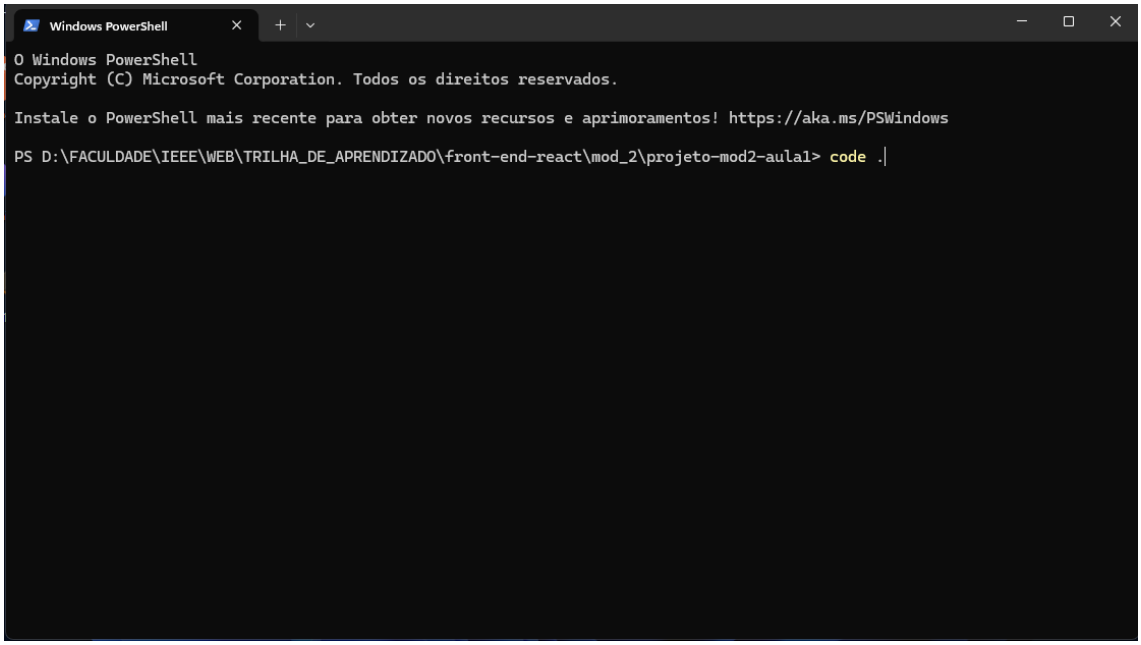

Imagem 13: Abrir projeto no VS Code

2. O projeto será aberto no VS Code.

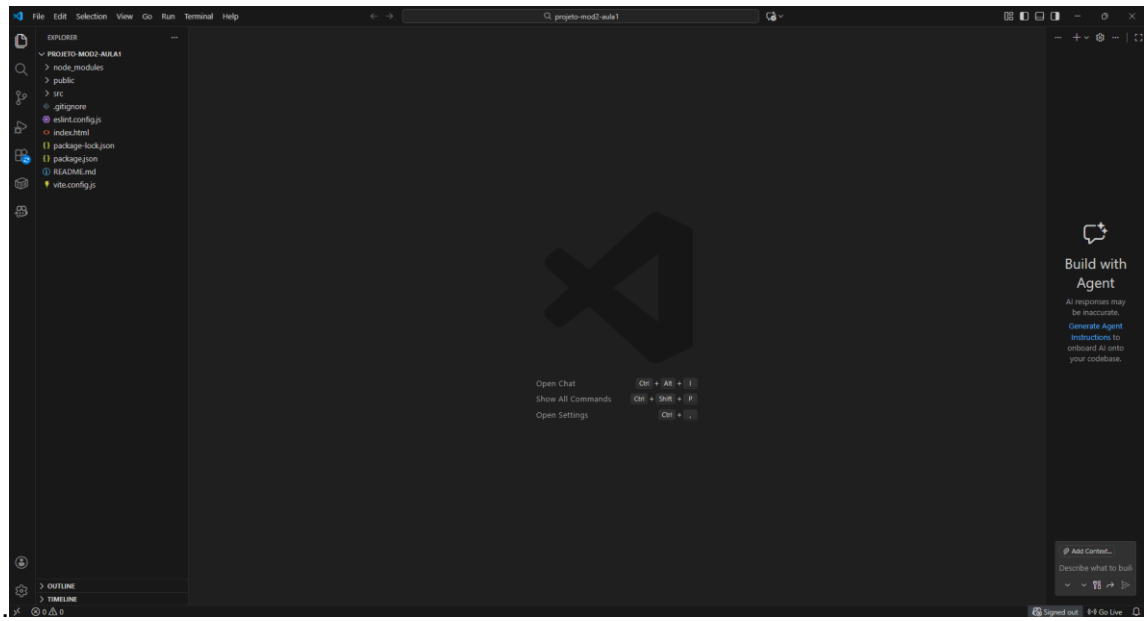

Imagem 14: Projeto no VS Code

Para maior comodidade nas próximas vezes, abra primeiro o projeto no VS Code e, em seguida, utilize o terminal integrado do próprio VS Code para executá-lo.

➢ Estrutura Básica de um Projeto React

Ao observar as pastas e os arquivos do projeto após a sua criação, é possível identificar a estrutura ilustrada na Imagem 15. As pastas são representadas pelos nomes que terminam com / e os arquivos são aqueles que não possuem essa terminação.

```
PROJETO-MOD2-AULA1/
├─ node_modules/
├─ public/
├─ src/
├─ .gitignore
├─ eslint.config.js
├─ index.html
├─ package-lock.json
├─ package.json
├─ README.md
└─ vite.config.js
```

Imagem 15: Estrutura de pastas e arquivos do Vite

Começando pela pasta node_modules/, ela é responsável por armazenar todas as bibliotecas e dependências que fazem o projeto funcionar. Essa pasta é criada automaticamente pelo gerenciador de pacotes e não deve ser versionada, pois as dependências do projeto já estão descritas no arquivo package.json.

Em seguida, tem-se a pasta public/, que é utilizada para armazenar arquivos estáticos, como imagens e ícones. Esses arquivos não passam pelo processo de build do Vite e são copiados diretamente para a raiz do projeto na versão de produção.

A pasta src/ é o diretório principal do código-fonte da aplicação. É nela que ficam a lógica do sistema, os componentes, os estilos e os arquivos principais do React, como main.jsx ou main.tsx e App.jsx ou App.tsx.

O arquivo .gitignore define quais arquivos ou pastas não devem ser versionados pelo Git. Normalmente, ele inclui arquivos desnecessários para o repositório ou que podem ser gerados automaticamente. Um exemplo importante é a pasta node_modules/, já que a lista de dependências do projeto está especificada no arquivo package.json.

O arquivo eslint.config.js contém as configurações do ESLint, uma ferramenta utilizada para padronização e análise de código. Esse arquivo define regras que ajudam a manter boas práticas de programação e a evitar erros comuns durante o desenvolvimento.

O arquivo index.html é o ponto de entrada da aplicação. Ele contém a estrutura HTML básica e a div onde a aplicação React será renderizada.

O arquivo package-lock.json é gerado automaticamente pelo npm e tem a função de garantir que as mesmas versões das dependências sejam instaladas em diferentes máquinas. Ele assegura consistência no ambiente de desenvolvimento e não deve ser editado manualmente.

O arquivo package.json descreve as configurações principais do projeto. Nele estão definidas informações como o nome do projeto, versão, scripts de execução, por exemplo `dev`, `build` e `preview`, além da lista de dependências e dependências de desenvolvimento.

O arquivo README.md é utilizado para documentar o projeto. Nele normalmente são descritos o objetivo da aplicação, instruções de instalação, comandos para execução e outras informações importantes para quem for utilizar ou contribuir com o projeto. Esse arquivo é escrito em Markdown e costuma ser exibido automaticamente em repositórios Git, como no GitHub.

O arquivo vite.config.js é responsável pela configuração do Vite. Nele podem ser definidos plugins, configurações de build, aliases de importação, portas do servidor de desenvolvimento, entre outras opções. Esse arquivo permite personalizar o comportamento do Vite de acordo com as necessidades do projeto.

➢ Componentizando elementos

Antes de mostrar a componentização, é importante entender o que é JSX.

JSX é uma extensão de sintaxe do JavaScript que permite escrever código com uma aparência semelhante ao HTML. Isso torna o código mais legível e familiar, pois não é necessário criar manualmente os elementos da interface por meio de funções JavaScript, como `createElement()`.

Antes de ser executado no navegador, o código JSX é transformado em JavaScript puro durante o processo de build.

Os componentes do React são criados por meio de funções que retornam um JSX. Para ter uma visualização melhor, apague o arquivo App.css, o conteúdo do arquivo index.css e tudo que estiver dentro de `<>...</>` no arquivo App.jsx.

Observe a estrutura de pastas na Imagem 16, onde será inserido o nosso componente de exemplo. Crie uma pasta chamada *components* dentro da pasta src e, dentro dela, crie os arquivos Button.jsx e Button.css.

```
PROJETO-MOD2-AULA1/
├─ node_modules/
├─ public/
├─ src/
  ├─ assets/
  ├─ components/
    ├─ Button.css
    └─ Button.jsx
  ├─ App.jsx
  └─ Main.css
```

Imagem 16: Arquivos de exemplo

Como já foi falado anteriormente, os componentes são criados por meio de funções JavaScript. Os parâmetros das funções no React são chamados de props (abreviação de *properties*, ou seja, propriedades) e são passados do componente pai para o componente filho. Na Imagem 17, vemos o componente Button, que recebe as props como parâmetro e retorna um JSX de um botão. Observe que o retorno em JSX está entre parênteses `()`. Dentro da tag `button`, temos entre as chaves, indicando que é um código JavaScript, `props.text`, onde `text` é o nome da propriedade que está sendo passada para esse botão. Após o código da função, temos o `export default Button`, indicando o nome com o qual esse componente será importado por outros componentes.

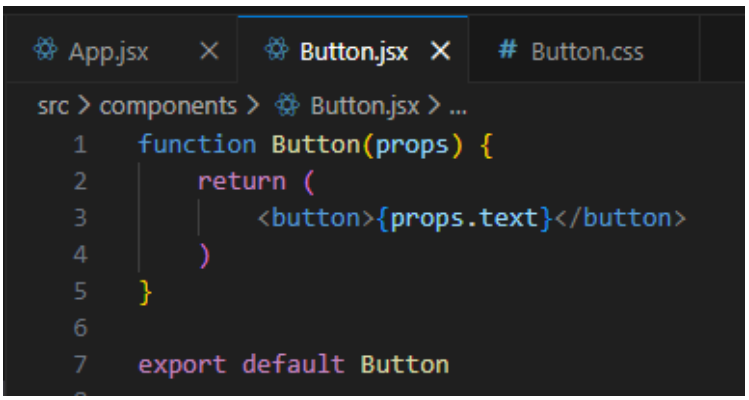

Imagem 17: Componente Button no react

Vamos agora para o arquivo App.jsx. Nele, podemos observar na primeira linha a importação do nosso componente Button. Para utilizar esse componente, foi escrito o código:

<Button text=”Botão 1” />

Observe na Imagem 18 que `text` é o nome da propriedade que havia sido definida na construção do componente Button. Um componente pode receber quantas propriedades forem necessárias.

```jsx
import Button from "./components/button"

function App() {


  return (
    <>
      <Button text="Botão 1" />
    </>
  )
}

export default App
```

Imagem 18: Utilizando o componente Button no componente App

Voltando para o navegador, como ilustrado na Imagem 19, vemos o nosso botão criado no canto superior esquerdo, com o texto que foi passado para o componente.

Imagem 19: Visualizando o componente Button no navegador

A vantagem de um componente é notada quando precisamos utilizá-lo mais de uma vez. Observe que é possível criar quantos botões forem necessários sem precisar reescrever o componente. Veja nas Imagens 20 e 21 que, no arquivo App.jsx, são criados vários botões.

```jsx
import Button from "./components/button"

function App() {


  return (
    <>
      <Button text="Botão 1" />
      <Button text="Botão 2" />
      <Button text="Botão 3" />
      <Button text="Botão 4" />
    </>
  )
}

export default App
```

Imagem 20: Utilizando mais de uma vez o componente Button

Ao retornar para o navegador, temos quatro botões criados, cada um com o seu próprio texto.

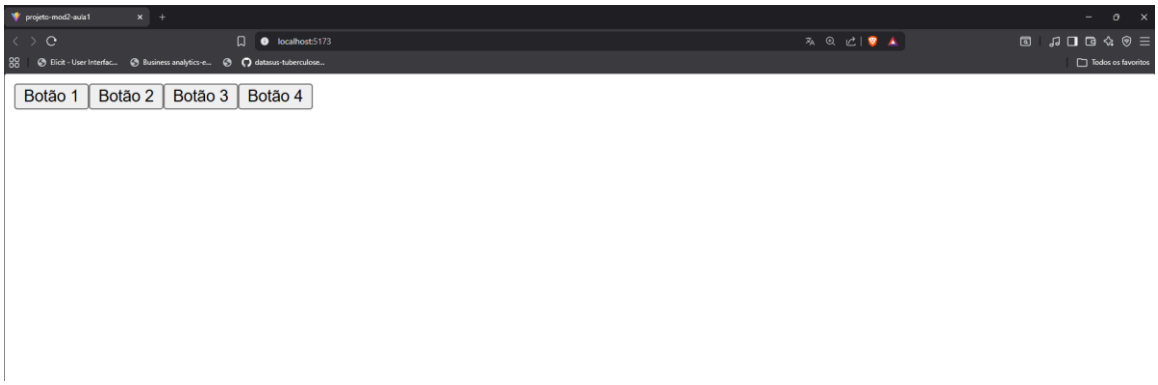

Imagem 21: Resultado no navegador

Para estilizar o componente, abra o arquivo Button.css, conforme ilustrado na Imagem 22, e insira os estilos. Nesse exemplo, foi criada a classe *button*, com a definição de suas propriedades, incluindo o efeito aplicado ao passar o mouse sobre o botão.

```
App.jsx    Button.jsx    # Button.css ×
src > components > # Button.css > .button
1   .button {
2       margin: 10px 5px;
3       padding: 3px 10px;
4       background-color: rgb(54, 54, 109);
5       color: white;
6       border: solid 1px black;
7       border-radius: 3px;
8   }
9
10  .button:hover {
11      background-color: rgb(101, 101, 197);
12      cursor: pointer;
13  }
```

Imagem 22: Estilos do componente Button

Para aplicar essa formatação no componente, é necessário acrescentar algumas coisas ao código. Primeiro, o arquivo de estilo é importado. Em seguida, a classe é aplicada ao botão por meio da propriedade `className`, passando o nome da classe, conforme ilustrado na Imagem 23.

```
App.jsx    Button.jsx ×    # Button.css
src > components > Button.jsx > ...
1   import './Button.css'
2
3   function Button(props) {
4       return (
5           <button className='button'>{props.text}</button>
6       )
7   }
8
9   export default Button
10
```

Imagem 23: Incluindo estilos no componente Button

Observe agora o resultado final no navegador, conforme ilustrado na Imagem 24.

Imagem 24: Resultado no navegador

Com isso, chegamos ao final do módulo 2, no qual aprendemos como criar um projeto, criar nossos componentes e estilizá-los de forma simples.